

Deploy a React + Vite Portfolio to Firebase Hosting for Free

This guide explains how to deploy a React + Vite portfolio to **Firebase Hosting**, connect a domain purchased from **Hostinger**, enable HTTPS, and redirect `www` to the main domain.

The final result will be:

```
https://malindasampath.com
  ↓
Your portfolio

https://www.malindasampath.com
  ↓
Redirect
  ↓
https://malindasampath.com
```

1. What We Are Going to Build

The deployment architecture is:

```
GitHub
  |
  | Source Code
  ▼
React + Vite Portfolio
  |
  | npm run build
  ▼
dist/
  |
  | firebase deploy
  ▼
Firebase Hosting
  |
  |— project-xxxxx.web.app
  |
  |— malindasampath.com
```



Firebase handles:

- Website hosting
- HTTPS/SSL
- CDN
- Deployment
- Custom domain connection

Hostinger is only used for:

- Domain registration
- DNS management

2. Requirements

You need:

- A React/Vite project
- A Google account
- A Firebase project
- A domain
- Access to your domain's DNS settings

In this example:

```
Domain:
malindasampath.com
```

3. Build the React Application

Before deploying, make sure the application builds successfully.

From the project directory:

```
npm run build
```

A successful build should create:

```
dist/  
├─ assets/  
├─ images/  
├─ icons/  
└─ index.html
```

The `dist` directory is the production version of your website.

Firebase Hosting will upload this directory.

4. Test the Docker Version Locally

If your project has a Dockerfile, you can test it locally first.

Example Dockerfile:

```
FROM node:20-alpine AS build  
  
WORKDIR /app  
  
COPY package*.json ./  
RUN npm ci  
  
COPY . .  
RUN npm run build  
  
FROM nginx:alpine  
  
COPY --from=build /app/dist /usr/share/nginx/html  
  
EXPOSE 80  
  
CMD ["nginx", "-g", "daemon off;"]
```

Build the image:

```
docker build -t portfolio .
```

Run it:

```
docker run -p 8080:80 portfolio
```

Then open:

```
http://localhost:8080
```

Important: Nginx listens on port `80` inside the container, so the correct mapping is:

```
-p 8080:80
```

5. Create a Firebase Project

Go to [Firebase Console](#).

Create a Firebase project or add Firebase to an existing Google Cloud project.

Once Firebase is enabled, Firebase will provide a project ID similar to:

```
project-c7396ba2-e46c-4c-17d7a
```

Keep this project ID. You will need it during deployment.

6. Install Firebase CLI

Firebase CLI allows you to deploy your website from the terminal.

Check whether it is installed:

```
firebase --version
```

Example:

```
15.24.0
```

If it is not installed, install it using npm:

```
npm install -g firebase-tools
```

7. Login to Firebase

On Cloud Shell or another terminal:

```
firebase login --no-localhost
```

Firebase will provide a URL.

Open that URL in your browser and complete authentication.

You will receive an authorization code.

Paste the code back into the terminal.

You should see:

```
Success! Logged in as your-email@gmail.com
```

8. Clone Your Portfolio

If your project is stored on GitHub:

```
git clone https://github.com/malinda-sampath/portfolio.git
```

Enter the directory:

```
cd portfolio
```

9. Install Dependencies

Run:

```
npm install
```

Then build:

```
npm run build
```

Verify:

```
ls dist
```

You should see your production files.

10. Initialize Firebase Hosting

Run:

```
firebase init hosting
```

Choose:

```
Use an existing project
```

Select your Firebase project.

For the public directory, enter:

```
dist
```

If asked whether this is a single-page application:

```
Yes
```

This is important for React applications because React handles client-side routes.

If asked whether to overwrite `index.html`:

No

11. Firebase Configuration

Your `.firebaserc` should point to the Firebase project:

```
{
  "projects": {
    "default": "project-c7396ba2-e46c-4c-17d7a"
  }
}
```

Your `firebase.json` should look similar to:

```
{
  "hosting": {
    "public": "dist",
    "ignore": [
      "firebase.json",
      "**/.*",
      "**/node_modules/**"
    ],
    "rewrites": [
      {
        "source": "**",
        "destination": "/index.html"
      }
    ]
  }
}
```

The important part is:

```
"public": "dist"
```

because Vite generates the production website inside `dist`.

12. Deploy the Website

Build first:

```
npm run build
```

Then deploy:

```
firebase deploy --only hosting
```

Or explicitly specify the project:

```
firebase deploy --only hosting --project project-c7396ba2-e46c-4c-17d7a
```

A successful deployment looks like:

✓ Deploy complete!

Hosting URL:

`https://project-c7396ba2-e46c-4c-17d7a.web.app`

Open the Hosting URL and confirm that your portfolio works.

13. If Firebase Hosting API Gives a 403 Error

Sometimes Firebase CLI can report:

```
Firebase Hosting API has not been used  
or it is disabled
```

Enable the Firebase Hosting API:

```
gcloud services enable firebasehosting.googleapis.com  
--project=project-c7396ba2-e46c-4c-17d7a
```

Verify:


```
gcloud services list --enabled
  --project=project-c7396ba2-e46c-4c-17d7a
  | grep firebasehosting
```

You should see:

```
NAME: firebasehosting.googleapis.com
```

Then retry:

```
firebase deploy --only hosting --project project-c7396ba2-e46c-4c-17d7a
```

14. Connect Hostinger Domain to Firebase

Now we connect:

```
malindasampath.com
```

to Firebase Hosting.

In Firebase:

```
Firebase Console
→ Hosting
→ Custom domains
→ Add custom domain
```

Enter:

```
malindasampath.com
```

Firebase will provide DNS records.

For the root domain, Firebase provided:

```
A
@
199.36.158.100
```

and:

```
TXT
@
hosting-site=project-c7396ba2-e46c-4c-17d7a
```

15. Configure DNS in Hostinger

Go to:

```
Hostinger
→ Domains
→ malindasampath.com
→ DNS / Nameservers
→ DNS Records
```

The final root-domain configuration is:

Type	Name	Content	TTL
A	@	199.36.158.100	300
TXT	@	hosting-site=project-c7396ba2-e46c-4c-17d7a	300

The old A record:

```
4.144.26.113
```

must be removed if Firebase tells you to remove it.

16. Verify DNS

You can check the A record:

```
nslookup malindasampath.com
```

Expected result:

```
Name: malindasampath.com  
Address: 199.36.158.100
```

Check the TXT record:

```
nslookup -type=TXT malindasampath.com
```

Expected:

```
hosting-site=project-c7396ba2-e46c-4c-17d7a
```

Once Firebase detects these records, the custom domain will become connected.

17. Wait for SSL Certificate

Firebase automatically creates an SSL certificate for the custom domain.

Firebase may show:

```
Minting certificate
```

This can take some time.

Once complete, the domain should show:

```
Connected
```

You don't need to manually purchase or install an SSL certificate.

18. Connect www.malindasampath.com

The root domain and `www` subdomain are treated separately.

The final setup we use is:

```
malindasampath.com
```

as the main domain.

Then:

```
www.malindasampath.com
```

redirects to:

```
malindasampath.com
```

In Firebase Hosting:

```
Hosting  
→ Custom domains  
→ Add custom domain
```

Enter:

```
www.malindasampath.com
```

For the redirect option, configure:

```
Redirect www.malindasampath.com  
to malindasampath.com
```

This gives you a single canonical URL.

19. Final DNS Configuration

The root domain should remain configured for Firebase.

For the `www` subdomain, use the DNS record Firebase requests when adding:

```
www.malindasampath.com
```

Do not blindly copy DNS records from another Firebase project.

Always use the values Firebase shows for your specific Hosting site.

20. Final Domain Behavior

After everything is configured:

Main domain

```
https://malindasampath.com
```

shows:

```
Your Portfolio
```

www domain

```
https://www.malindasampath.com
```

redirects to:

```
https://malindasampath.com
```

Firebase default domain

```
https://project-c7396ba2-e46c-4c-17d7a.web.app
```

also serves the portfolio.

21. Important Vite Asset Rule

Vite projects treat the `public` directory specially.

For example, if you have:

```
public/  
└─ icons/  
   └─ WS02-Logo-White.png
```

you should reference it as:

```
/icons/WS02-Logo-White.png
```

NOT:

```
/public/icons/WS02-Logo-White.png
```

Vite copies the contents of `public` directly into the root of `dist`.

Therefore:

```
public/icons/WS02-Logo-White.png
```

becomes:

```
dist/icons/WS02-Logo-White.png
```

and the browser URL is:

```
/icons/WS02-Logo-White.png
```

22. Updating the Website

Whenever you change your portfolio:

```
git pull
```

or make your code changes.

Then:

```
npm install
```

if dependencies changed.

Build:

```
npm run build
```

Deploy:

```
firebase deploy --only hosting
```

That's all.

You don't need to manually upload files.

23. Recommended Git Workflow

A simple workflow is:

```
Make changes
  ↓
git add .
  ↓
git commit
  ↓
git push
  ↓
npm run build
  ↓
firebase deploy --only hosting
```

Later, you can automate the deployment using GitHub Actions so that every push to `main` automatically deploys your portfolio.

24. Troubleshooting Checklist

Firebase shows "Site Not Found"

Check:

```
firebase deploy --only hosting
```

Make sure deployment finishes with:

```
✓ Deploy complete!
```

Also check:

```
ls dist
```

and confirm that `index.html` exists.

Custom domain doesn't work

Check:

```
nslookup malindasampath.com
```

The A record should point to the Firebase address provided by Firebase.

Also check:

```
nslookup -type=TXT malindasampath.com
```

The Firebase verification TXT record should exist.

www doesn't work

Check that:


```
www.malindasampath.com
```

has been added as a custom domain in Firebase.

Then configure it to redirect to:

```
malindasampath.com
```

HTTPS isn't working

Wait for Firebase to finish:

```
Minting certificate
```

Do not manually purchase an SSL certificate.

Images return 404

Check whether the files are inside:

```
public/
```

If they are, don't include `/public` in the browser path.

Example:

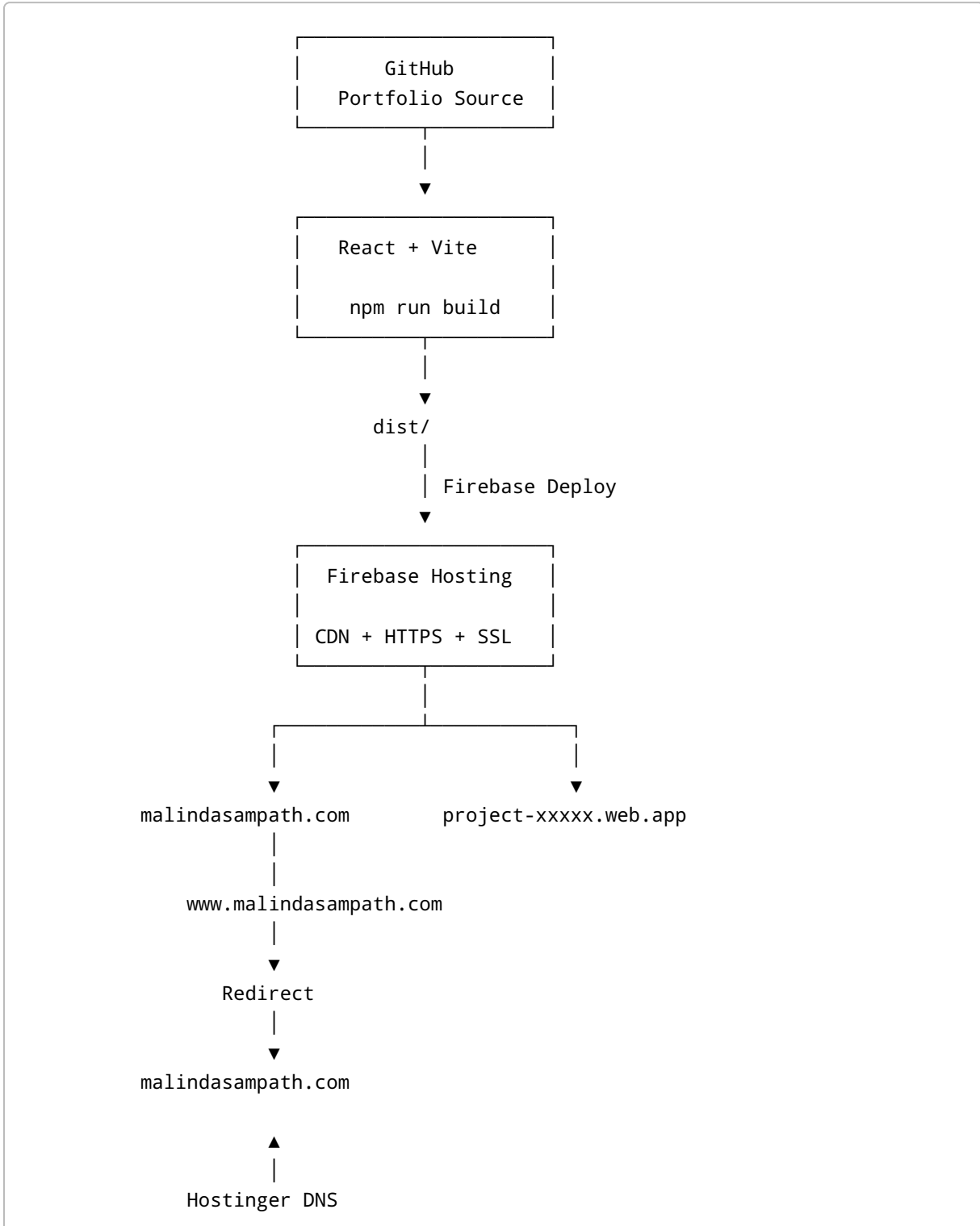
```
public/images/profile.png
```

should be:

```
/images/profile.png
```

25. Final Architecture

Your final portfolio infrastructure is:



26. Final Checklist

Before considering the deployment complete:

- [x] React/Vite project builds successfully
- [x] `dist/` directory created
- [x] Firebase project created
- [x] Firebase CLI installed
- [x] Firebase authentication completed
- [x] Firebase Hosting enabled
- [x] Firebase Hosting API enabled
- [x] `firebase.json` configured
- [x] `.firebaserc` configured
- [x] Portfolio deployed
- [x] Firebase `.web.app` URL works
- [x] Hostinger DNS configured
- [x] Firebase TXT verification completed
- [x] Firebase A record configured
- [x] SSL certificate created
- [x] `malindasampath.com` connected
- [x] `www.malindasampath.com` configured
- [x] `www` redirects to `malindasampath.com`

Final URLs

Primary portfolio:

`https://malindasampath.com`

www redirect:

`https://www.malindasampath.com`

Firebase Hosting URL:

`https://project-c7396ba2-e46c-4c-17d7a.web.app`

27. Cost

Firebase Hosting has a no-cost tier, so a small personal portfolio with low traffic can generally be hosted without paying for hosting.

However, Google Cloud/Firebase pricing and quotas can change, so always check the current Firebase pricing page before assuming a workload will remain completely free.

For a normal personal portfolio with modest traffic, this setup is a very practical low-cost/free option.

28. Summary

The simplest way to remember the entire process is:

1. Build React app
- ↓
2. npm run build
- ↓
3. Get dist/
- ↓
4. Firebase Hosting
- ↓
5. firebase deploy
- ↓
6. Connect domain
- ↓
7. Configure Hostinger DNS
- ↓
8. Wait for SSL
- ↓
9. Connect www
- ↓
10. Redirect www → root domain

Your production portfolio is now hosted on Firebase, while **Hostinger remains your domain/DNS provider**.